

lecture_18

September 10, 2026

1 Lecture 18: Constrained Optimization — Interior-Point Method

{note} Lecture 17 introduced the Barrier Method: add a logarithmic wall to prevent iterates from leaving the feasible set, then reduce the wall height until the barrier solution converges to the KKT point. The approach works, but each outer iteration requires solving an unconstrained subproblem to full convergence - which is itself an iterative process. The Interior-Point Method (IPM) removes this nested structure. Rather than waiting for an inner solver to converge, it applies a single Newton step directly to the full KKT system - primal variables, slacks, and multipliers all at once - while maintaining strict feasibility via the same logarithmic barrier. The result is a method with polynomial-time convergence guarantees that underpins virtually every industrial-grade NLP solver.

Learning Objectives

By the end of this notebook, you will be able to: 1. State the Karush–Kuhn–Tucker (KKT) conditions and identify what each condition requires of a constrained optimum. 2. Formulate the primal-dual KKT system for the logarithmic barrier subproblem and derive the Newton step equations. 3. Trace one Newton step of the Interior-Point Method by hand and explain how it differs from the Barrier Method in computational structure.

Prerequisites: NLP Principles (Lecture 12); Gradient Descent, Newton’s Method, BFGS (Lectures 13–15); Penalty Method (Lecture 16); Barrier Method (Lecture 17); calculus (partial derivatives); basic linear algebra (solving 4×4 linear systems).

Estimated time: 90 minutes (including in-class exercises)

1.1 Optimality Conditions — KKT

Consider the standard-form NLP:

$$\min_x f(x) \quad \text{subject to} \quad g_i(x) \leq 0, \quad i = 1, \dots, m$$

where $f, g_i : \mathbb{R}^n \rightarrow \mathbb{R}$ are continuously differentiable. The **Lagrangian** is:

$$\mathcal{L}(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x)$$

If x^* is a local minimum and a constraint qualification holds (e.g., LICQ), then there exist multipliers $\lambda^* \in \mathbb{R}^m$ such that the following **Karush–Kuhn–Tucker (KKT) conditions** are satisfied:

Condition	Statement	Interpretation
Stationarity	$\nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(x^*) = \mathbf{0}$	Gradient of objective balanced by gradients of active constraints
Primal feasibility	$g_i(x^*) \leq 0 \quad \forall i$	Solution lies within the feasible region
Dual feasibility	$\lambda_i^* \geq 0 \quad \forall i$	Multipliers are non-negative for inequality constraints
Complementary slackness	$\lambda_i^* g_i(x^*) = 0 \quad \forall i$	A constraint is either active ($g_i = 0$) or its multiplier is zero ($\lambda_i = 0$)

For a **convex** NLP (convex f , convex g_i), the KKT conditions are both necessary and sufficient for global optimality. All methods in Lectures 16–18 seek a point that satisfies these four conditions.

1.2 Constrained Optimization — Framework

The key insight behind the Constrained Optimization Frameworks is simple: if violating a constraint is made sufficiently expensive, the optimizer will avoid violations on its own. Lectures 16–18 introduce three algorithms that compute a KKT point by transforming the constrained problem into a sequence of tractable subproblems.

Method	Core Idea	Subproblem Type	Feasibility of Iterates	Convergence
Penalty Method	Append a penalty for constraint violation	Unconstrained minimization	Iterates are infeasible	Exact KKT point recovered only in the limit; linear rate
Barrier Method	Append a barrier that blows up near constraint boundary	Unconstrained minimization	Iterates are strictly feasible	Exact KKT point recovered only in the limit; linear rate

Method	Core Idea	Subproblem Type	Feasibility of Iterates	Convergence
Interior-Point Method	Solve KKT system directly with barrier modification; Newton steps inside feasible region	Equality-constrained Newton system	Iterates are strictly feasible	Exact KKT point recovered only in the limit; polynomial-time; superlinear rate

1.3 3. Interior-Point Method

Recall that the Barrier Method solves a sequence of unconstrained subproblems $\min_x B(x, \mu) = f(x) + \mu \sum_{i=1}^m \ln(-1/g_i(x))$ with progressively smaller μ , where the optimality condition for each subproblem is $\nabla B(x, \mu) = \mathbf{0}$. The Interior-Point Method starts from exactly this subproblem but reformulates its optimality condition by introducing **slack variables** $s_i = -g_i(x); s_i > 0$ and **multipliers** $\lambda_i = \mu/s_i; \lambda_i > 0$. Substituting into $\nabla B(x, \mu) = \mathbf{0}$ renders the **primal-dual system** of equations $F(x, s, \lambda) = \mathbf{0}$.

$$F(\mathbf{X}) = \begin{cases} \nabla f(x) + \sum_{i=1}^m \lambda_i \nabla g_i(x) = \mathbf{0} & \text{(stationarity)} \\ g_i(x) + s_i = 0, \quad i = 1, \dots, m & \text{(primal feasibility)} \\ \lambda_i s_i - \mu = 0, \quad i = 1, \dots, m & \text{(relaxed complementarity)} \end{cases} ; \quad X = (x, s, \lambda)$$

In practice, the Interior-Point Method iteratively solves $F(X^{(k)}) = \mathbf{0}$ with a progressively smaller barrier parameter $\mu^{(k+1)} = \theta \cdot \mu^{(k)}; \theta \in (0, 1)$ until the residual norm falls below a pre-defined tolerance $\|F^{(k)}\| < \varepsilon$. Thus, as $\mu \rightarrow 0$, the primal-dual system converges to the KKT system.

However, finding the solution of the primal-dual system of equations is a nonlinear problem. To this end, the Interior-Point Method employs Newton's Method. Recall that Newton's Method (Lecture 14) solves $\nabla f(x) = \mathbf{0}$ by starting at an initial value $x^{(0)}$ and iteratively computing $x^{(k+1)} = x^{(k)} - [\nabla^2 f(x^{(k)})]^{-1} \nabla f(x^{(k)})$ until convergence. The Interior-Point Method applies exactly the same idea to the primal-dual system by starting from a strictly feasible $\mathbf{X}^{(0)}$ with $s^{(0)} > 0$ and $\lambda^{(0)} > 0$ and computing $\mathbf{X}^{(k+1)} = \mathbf{X}^{(k)} - \alpha^{(k)} [\nabla F(\mathbf{X}^{(k)})]^{-1} F(\mathbf{X}^{(k)})$. However, unlike Newton's Method for unconstrained minimization, the Interior-Point Method requires a **step size** $\alpha^{(k)} \in (0, 1]$ to prevent s and λ from going negative, which would take the iterate outside the feasible region where $\ln(-1/g_i)$ is undefined. This step size is chosen by the **fraction-to-boundary rule** ($\tau \in (0.9, 1)$):

$$\alpha_{\max} = \min \left(1, \tau \cdot \min_i \left\{ \frac{-s_i}{\Delta s_i} : \Delta s_i < 0 \right\}, \tau \cdot \min_i \left\{ \frac{-\lambda_i}{\Delta \lambda_i} : \Delta \lambda_i < 0 \right\} \right)$$

The key difference between the Barrier Method and Interior-Point Method is computational: the Barrier Method requires many unconstrained iterations per μ value, whereas the Interior-Point Method reaches the same point in a single Newton step, with multipliers λ_i available explicitly

throughout rather than recovered post-hoc as μ/s_i . This combination of polynomial-time convergence and quadratic local convergence underpins virtually every industrial-grade NLP solver (IPOPT, Gurobi, Knitro).

{tip} The fraction-to-boundary step size α must be computed separately for s and λ : scan only those components where $\Delta s_i < 0$ (or $\Delta \lambda_i < 0$) - components moving away from zero need no cap.

{note} The Interior-Point Method approaches a KKT point from strictly inside the feasible region - iterates always satisfy $g_i(x) < 0$ - following the same central path as the Barrier Method. What differs is the computational structure: a single Newton step on the primal-dual system $F(\mathbf{X}) = \mathbf{0}$ simultaneously updates primal variables, slacks, and multipliers, achieving quadratic convergence per μ reduction.

1.4 In-Class Exercises

1.4.1 Exercise 1

MTC Chennai is optimizing a demand-responsive feeder service on the Tambaram–Chromepet corridor. The daily operating cost depends on fleet size n (vehicles) and scheduled headway h (minutes):

$$C(n, h) = (n - 5)^2 + 2(h - 3)^2$$

The depot has a combined resource constraint: $n + h \leq 7$.

Introduce slack $s = 7 - n - h > 0$ and multiplier $\lambda > 0$. Substituting into $\nabla B(n, h, \mu) = \mathbf{0}$ and collecting terms yields the primal-dual system $F(\mathbf{X}) = \mathbf{0}$, where $\mathbf{X} = (n, h, s, \lambda)$:

$$F(\mathbf{X}) = \begin{pmatrix} 2(n - 5) + \lambda \\ 4(h - 3) + \lambda \\ n + h + s - 7 \\ \lambda s - \mu \end{pmatrix} = \mathbf{0}$$

Applying Newton's Method to $F(\mathbf{X}) = \mathbf{0}$ gives the Newton step $\Delta \mathbf{X}^{(k)} = -[\nabla F(\mathbf{X}^{(k)})]^{-1} F(\mathbf{X}^{(k)})$, where ∇F is the matrix of partial derivatives of F :

$$\nabla F(\mathbf{X}) = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 0 & 4 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & \lambda & s \end{bmatrix}$$

1. Set $\mu = 1$ and starting point $(n, h, s, \lambda) = (3, 3, 1, 1)$. Verify this point is strictly feasible: $s > 0$, $\lambda > 0$, and primal feasibility $n + h + s = 7$.
2. Evaluate all four residuals $F(3, 3, 1, 1)$.

3. Compute the Newton step $\Delta \mathbf{X}$ by solving the linear system $\nabla F(\mathbf{X}) \Delta \mathbf{X} = -F(\mathbf{X})$ at the starting point. (*Hint: start from row 4: $\Delta s + \Delta \lambda = 0$, then eliminate $\Delta \lambda$ from rows 1–2.*) Verify that $\Delta s = -5/7$, $\Delta n = 8/7$, $\Delta h = -3/7$, $\Delta \lambda = 5/7$.
4. Apply the update $\mathbf{X}^{(1)} = \mathbf{X}^{(0)} + \alpha \Delta \mathbf{X}$ with full step $\alpha = 1$. Confirm that:
 - Strict feasibility is maintained: $s^{(1)} = 2/7 > 0$, $\lambda^{(1)} = 12/7 > 0$.
 - Primal feasibility holds exactly: $n^{(1)} + h^{(1)} + s^{(1)} = 7$.
 - Stationarity residuals r_1, r_2 are exactly zero after one Newton step.
 - Only the complementarity residual $r_4 = \lambda^{(1)} s^{(1)} - \mu = 24/49 - 1 \neq 0$ remains.
5. Explain what this observation tells you about the Newton step: which KKT conditions does one step satisfy exactly, and which does it only partially satisfy?

Newton Step Table

Quantity	Symbolic	Value
Starting point (n, h, s, λ)	—	(3, 3, 1, 1)
$r_1 = 2(n - 5) + \lambda$		
$r_2 = 4(h - 3) + \lambda$		
$r_3 = n + h + s - 7$		
$r_4 = \lambda s - \mu$		
Δs	$-5/7$	
Δn	$8/7$	
Δh	$-3/7$	
$\Delta \lambda$	$5/7$	
New point $(n^{(1)}, h^{(1)}, s^{(1)}, \lambda^{(1)})$	$(29/7, 18/7, 2/7, 12/7)$	
$r_1^{(1)} = 2(n^{(1)} - 5) + \lambda^{(1)}$	0	
$r_4^{(1)} = \lambda^{(1)} s^{(1)} - \mu$	$24/49 - 1$	

1.4.2 Exercise 2

An NHAH corridor study models total system travel time across two parallel links — the Mumbai–Pune Expressway (v_1 , thousand veh/hr) and old NH48 (v_2 , thousand veh/hr):

$$T(v_1, v_2) = 3v_1^2 - 2v_1v_2 + 2v_2^2 - 14v_1 - 12v_2 + 50$$

A combined capacity constraint limits total flow: $v_1 + v_2 \leq 5$. The Hessian of T is $\nabla^2 T = \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix}$ and the constraint gradient is $\nabla g = (1, 1)^\top$.

1. Introduce slack $s = 5 - v_1 - v_2 > 0$ and multiplier $\lambda > 0$. Write the full primal-dual system $F(v_1, v_2, s, \lambda) = \mathbf{0}$ and the matrix of partial derivatives $\nabla F(\mathbf{X})$, following the same structure as Exercise 1.
2. Using starting point $(v_1, v_2, s, \lambda) = (1, 2, 2, 1)$ and $\mu = 1$, evaluate the residuals $F(\mathbf{X})$.
3. Compare the three methods on this problem by filling in the table below. Use the Penalty results from Lecture 16 and the Barrier results from Lecture 17.

Three-Method Comparison: NHA1 Parallel Links

Method	Parameter	v_1	v_2	$v_1 + v_2$	Implicit $\hat{\lambda}$	Feasible?
Penalty	$\rho = 1$	3.294	4.059	7.353	0.571	No
Penalty	$\rho = 5$	2.667	3.222	5.889	1.053	No
Penalty	$\rho = 25$	2.378	2.838	5.216	1.266	No
Penalty	$\rho = 125$	2.305	2.740	5.045	1.319	No
Barrier	$\mu = 2$	2.025	2.367	4.392	6.582	Yes
/ IP						
Barrier	$\mu = 1$	2.214	2.618	4.832	5.954	Yes
/ IP						
Barrier	$\mu = 0.1$	2.278	2.704	4.983	5.739	Yes
/ IP						
Barrier	$\mu = 0.01$	2.285	2.713	4.998	5.717	Yes
/ IP						
KKT (true)	—	16/7	19/7	5	40/7	Active

4. The true multiplier is $\lambda^* = 40/7 \approx 5.714$. Answer the following:

- Which method has implicit multiplier estimates closer to λ^* for moderate parameter values?
- For the Penalty Method, the implicit multiplier $\hat{\lambda} = \rho \cdot \max(0, v_1 + v_2 - 5)$ is well below λ^* even at $\rho = 125$. Why?
- For the Barrier / IP trajectory, the implicit multiplier $\hat{\lambda} = \mu/s$ overshoots λ^* at large μ and converges from above. Why does it overshoot?

1.5 Take-Away Exercises

1.5.1 Exercise 1

Delhivery is minimizing the daily operating cost at its Chennai cross-dock facility. The cost depends on daily throughput p (tonnes/day) and staffing level w (workers):

$$C(p, w) = 2p^2 - 4pw + 3w^2 - 12p - 6w + 50$$

A combined budget and shift constraint limits $p + 2w \leq 14$. Introduce slack $s = 14 - p - 2w > 0$ and multiplier $\lambda > 0$.

1. Write the primal-dual system $F(p, w, s, \lambda) = \mathbf{0}$ for barrier parameter μ . Identify the Hessian of C and the gradient of the constraint.
2. Using starting point $(p, w, s, \lambda) = (4, 3, 4, 1)$ — verify it satisfies $p + 2w + s = 14$ — and $\mu = 1$, evaluate all residuals.
3. Compute the Newton step $\Delta \mathbf{X} = -[\nabla F(\mathbf{X})]^{-1} F(\mathbf{X})$ by substituting numerical values. Apply the update $\mathbf{X}^{(1)} = \mathbf{X}^{(0)} + \alpha \Delta \mathbf{X}$ with $\alpha = 1$ if strict feasibility is maintained.

4. Compare your Interior-Point result with the Penalty and Barrier results from Lectures 16 and 17 by filling in the table below. What do all three methods agree on?

Three-Method Comparison: Delhivery Cross-Dock

Method	Parameter	p	w	$p + 2w$	Implicit $\hat{\lambda}$	Feasible?
Penalty	$\rho = 1$					
Penalty	$\rho = 25$					
Barrier / IP	$\mu = 1$	5.998	3.855	13.708	3.430	Yes
Barrier / IP	$\mu = 0.01$	6.104	3.946	13.997	3.369	Yes
KKT (true)	—	116/19	75/19	14	64/19	Active

5. The Delhivery operations manager asks: “If our combined budget-shift limit increases from 14 to 15, how much does daily cost decrease?” Use $\lambda^* = 64/19$ to answer, and confirm by computing C at the new optimum.

1.5.2 Exercise 2

BMTC is optimizing its demand-responsive feeder service in Bengaluru. The total daily operating cost depends on fleet size n (vehicles) and headway h (minutes):

$$C(n, h) = 2n^2 + 3h^2 + 2nh - 20n - 30h + 100$$

A depot capacity constraint limits $n \leq 2$.

1. Introduce slack $\sigma = 2 - n > 0$ and multiplier $\nu > 0$. Write the primal-dual system $F(n, h, \sigma, \nu) = \mathbf{0}$ for barrier parameter μ .
2. Implement the full Interior-Point Method in Python. At each outer iteration, perform Newton steps on the (n, h, σ, ν) system until $\|F\| < 10^{-8}$, then reduce μ by factor $\theta = 0.1$. Start from $(n, h, \sigma, \nu) = (1.5, 4.5, 0.5, 1.0)$ and $\mu_0 = 2$. Stop when $\mu < 10^{-4}$.
3. Report for each outer iteration: $n, h, \sigma, \nu, \|F\|$ at convergence, and the implicit multiplier $\hat{\nu} = \mu/\sigma$. How many Newton steps does each outer iteration require?
4. Produce a three-method comparison plot in Python. On a single figure with two panels:
 - *Left panel:* Plot $n^*(\text{parameter})$ vs. parameter index for Penalty ($\rho = 1, 5, 25, 125$), Barrier ($\mu = 2, 1, 0.1, 0.01$), and Interior-Point (same μ sequence). Mark the true KKT value $n^* = 2$ as a horizontal dashed line.
 - *Right panel:* Plot implicit multiplier $\hat{\nu}$ vs. parameter index for each method. Mark $\nu^* = 10/3$ as a horizontal dashed line.
5. Interpret $\nu^* = 10/3$ for the BMTC operations manager. Is this interpretation consistent across all three methods? Should it be?

1.6 Further Reading

- Nocedal, J. and Wright, S.J. (2006). *Numerical Optimization* (2nd ed.). Springer — Chapter 19 (interior-point methods for nonlinear programming, primal-dual systems, the central path).
- Bazaraa, M.S., Sherali, H.D., and Shetty, C.M. (2006). *Nonlinear Programming: Theory and Algorithms* (3rd ed.). Wiley — Chapter 12 (barrier and interior-point methods).
- Wächter, A. and Biegler, L.T. (2006). On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1), 25–57. (The paper behind IPOPT — the solver used inside `scipy.optimize.minimize` with `method='trust-constr'`.)
- SciPy documentation: `scipy.optimize.minimize` with `method='trust-constr'` — docs.scipy.org